



Comments from VB

Victorio's comments 2026/01/29

- He likes UCs, ADRs, Business Glossary
- Provide a simplified sequence diagram for blue UCs
 - The detailed sequence diagrams can be a .md file,
 - Manage the diagrams in Git, but deliver them as a ZIP
- Updated
 - ERS-ERE contract
 -
- We need to see that the cases in the PRD are covered by the REST API
 - The correspondence of REST API operations
 - Provide a table/diagram tracing
 - Client API to ERE
 - LinkCuration API to ERE
- Detailed definition of rest API can be omitted in this delivery

UC-W1

UC-W-01 — Resolve Entity Mention

Use Case Level

White (Summary)

Primary Actor

API Client (e.g. TED-SWS, Link Curation Backend, or any authorised originator)

Scope

Entity Resolution System (ERSys)

Stakeholders and Interests

- **API Client:** requires a stable, authoritative canonical identifier for each submitted entity mention, suitable for immediate downstream processing and later reconciliation.
- **Downstream consumers:** depend on canonical identifiers as stable proxies for managed identities, regardless of subsequent resolution or curation activity.
- **ERS operators:** require predictable, bounded API behaviour, strict idempotency, and isolation from engine-level failures.

Brief Description

The API Client submits an Entity Mention to ERSys in order to obtain a canonical identifier representing the managed identity under which the mention is governed.

For each valid Entity Mention, ERSys deterministically derives and returns a canonical identifier according to a standardised derivation rule. The canonical identifier is returned synchronously and within bounded time, independently of the availability or completion of automated resolution processing.

Automated resolution and candidate evaluation may occur asynchronously and may subsequently influence the governance state of the mention through Resolution Decisions, but they do not delay or block the assignment of a canonical identifier.

Preconditions

- The API Client is authorised to call the resolution API.
- The request includes the required correlation information, including an originator identifier and a request identifier.
- The Entity Mention payload conforms to supported formats and entity types.

Success Guarantee (Postconditions)

For every valid Entity Mention request:

- ERSys registers the request and enforces idempotency based on the originator and request identifiers.

- ERSys assigns a canonical identifier derived deterministically according to the applicable derivation rule.
- ERSys returns the canonical identifier correlated to the submitted request.
- A Resolution Decision exists for the Entity Mention, capturing the current governance state, including any available automated assessment.

Minimal Guarantees (On Failure)

- If the request is syntactically invalid or violates mandatory validation rules, ERSys returns an explicit error response and does not assign a canonical identifier.
- If internal dependencies required for request registration are unavailable, ERSys returns an explicit service error.
- Failures or delays in automated resolution processing do not prevent canonical identifier assignment or response delivery.

Main Success Scenario

1. The API Client submits a resolve request containing a single Entity Mention and the required correlation identifiers.
2. ERSys validates the request and registers it, enforcing idempotency.
3. ERSys deterministically derives the canonical identifier for the Entity Mention.
4. ERSys returns the canonical identifier to the API Client within bounded time.
5. ERSys may initiate or continue asynchronous resolution processing to evaluate candidate alignments and update the associated Resolution Decision.

Extensions

1a. Invalid or malformed request

ERSys rejects the request and returns a validation error describing the issue.

1b. Unsupported entity type or payload format

ERSys rejects the request and returns an explicit error indicating the unsupported type or format.

2a. Repeated request (idempotent replay)

If a request with the same originator and request identifiers is submitted again, ERSys does not create a new identity and returns the canonical identifier previously assigned to the request.

Special Requirements

- The resolve operation is idempotent with respect to request registration and canonical identifier assignment.
- Canonical identifier derivation is deterministic and standardised; the specific derivation mechanism is not exposed via this use case.
- Automated resolution, candidate ranking, and confidence assessment are asynchronous and do not affect the synchronous guarantees of this use case.

Technology and Data Variations

Entity Mention payloads may vary in format and structure, subject to validation and supported entity types.

Frequency of Occurrence

High; expected to be invoked repeatedly as part of automated ingestion and synchronisation pipelines.

Open Issues

None at this level. Detailed error taxonomies and resolution outcome representations are defined in lower-level use cases and interface specifications.

-

UC-B1.1

UC-B-1.1 — Resolve Entity Mention via ERS API

Use Case Level

Blue (User-goal)

Primary Actor

API Client (e.g. TED-SWS, Link Curation Backend, or any authorised originator)

Scope

Entity Resolution System (ERSys) — Public API (ERS)

Supporting Actors

- Canonical Entity Registry (authoritative projection managed by ERS)
- Resolution Decision Store (authoritative governance store managed by ERS)
- Messaging Middleware (for forwarding resolution requests)

Stakeholders and Interests

- **API Client:** wants a canonical identifier for a submitted entity mention with predictable behaviour, strict idempotency, and bounded response time.
- **ERS operators:** require isolation from engine delays, deterministic behaviour, and resilience to partial failures.
- **Downstream consumers:** rely on canonical identifiers as stable references for linking and downstream processing.

Brief Description

This use case describes the synchronous processing performed by ERS when an external client submits a request to resolve a single Entity Mention.

ERS processes the request by registering it as an authoritative record, deterministically deriving a canonical identifier, creating or updating the corresponding Resolution Decision with a default governance state, inserting the canonical assignment into the Canonical Entity Registry, and forwarding the request for asynchronous resolution processing.

The canonical identifier is returned to the client within bounded time. Automated resolution is fully decoupled from the API response and is refined separately in UC-B-1.2.

Trigger

The API Client submits a resolve request for a single Entity Mention.

Preconditions

- The API Client is authorised to call the ERS API.

- The request contains the required correlation fields (originatorId, originatorRequestId), entityType, payload, and payloadFormat.
- ERS maintains:
 - an append-only Request Registry for request registration and idempotency enforcement,
 - a Resolution Decision Store for governance state,
 - a Canonical Entity Registry representing the current canonical state.

Success Guarantee (Postconditions)

For a valid Entity Mention request:

- Exactly one canonical identifier is deterministically derived and assigned.
- A Resolution Decision is created or updated with a default governance state.
- The canonical assignment is recorded in the Canonical Entity Registry.
- A resolution request is forwarded asynchronously for automated processing.
- The canonical identifier is returned to the API Client within bounded time.

Minimal Guarantees (On Failure)

- If the request is invalid, ERS returns an explicit error response and does not assign a canonical identifier.
- If request registration or persistence fails, ERS returns a service error.
- Failures or delays in asynchronous resolution processing do not affect the synchronous response.

Main Success Scenario

1. The API Client submits a resolve request containing a single Entity Mention and correlation identifiers.
2. ERS validates the request and the Entity Mention.
3. ERS registers the request and enforces idempotency based on (originatorId, originatorRequestId).
4. ERS deterministically derives the canonical identifier for the Entity Mention.
5. ERS creates or updates a Resolution Decision with a default governance state.
6. ERS records the canonical assignment in the Canonical Entity Registry.
7. ERS publishes the resolution request for asynchronous processing.
8. ERS returns the canonical identifier to the API Client.

Extensions

2a. Invalid Entity Mention

If required fields are missing, the payload is malformed, or the payload format is unsupported, ERS rejects the request and returns a validation error.

2b. Unsupported entity type

ERS returns an error response with category UNSUPPORTED_ENTITY_TYPE.

3a. Idempotency conflict

If the same (originatorId, originatorRequestId) is submitted with a different payload, ERS returns an IDEMPOTENCY_CONFLICT error and does not change any existing state.

4a. Repeated request (idempotent replay)

If the same (originatorId, originatorRequestId) is submitted again with an identical payload, ERS returns the previously assigned canonical identifier without reprocessing.

5a. Rate limit exceeded

ERS rejects the request with a rate-limit error.

Special Requirements

- Singular semantics: each request processes exactly one Entity Mention.
- Determinism: canonical identifier derivation is deterministic and standardised.
- Idempotency: (originatorId, originatorRequestId) uniquely identifies a request.
- Bounded response time: synchronous processing does not wait for automated resolution.

Technology and Data Variations

- Supported payload formats and entity types are configuration-driven.
- Automated resolution capabilities are provided via connected Entity Resolution Engines and refined in UC-B-1.2.

Frequency of Occurrence

High; expected to be invoked repeatedly by ingestion pipelines and integration services.

Notes / Business Rules

- Confidence values, if computed, are internal governance signals only and are not exposed to API clients.

Open Issues

- Final error code taxonomy for external APIs.
- Whether originator identifiers are redundantly echoed in successful responses.

UC-B1.1 diagram

Originator asks for a resolution

[Diagram](#)

sequenceDiagram

```
participant Originator
participant EntityResolutionRESTAPI
participant ERS
participant RequestRegistry
participant CanonicalEntityRegistry
participant ResolutionDecisionStore
participant MessagingMiddleware

Originator->>EntityResolutionRESTAPI: "POST Resolve Entity
Mention(originatorId,originatorRequestId) "
EntityResolutionRESTAPI->>ERS: "Validate Request And Parse Payload"
alt InvalidRequest
ERS->>EntityResolutionRESTAPI: "400 Bad Request"
EntityResolutionRESTAPI->>Originator: "400 Bad Request"
else ValidRequest
ERS->>ERS: "Create Entity Mention And Assign Mention URI"
ERS->>RequestRegistry: "Upsert Request Record With Entity Mention"
alt ExistingRequestDifferentPayload
RequestRegistry-->>ERS: "Idempotency Conflict"
ERS->>EntityResolutionRESTAPI: "409 Conflict"
EntityResolutionRESTAPI->>Originator: "409 Conflict"
else IdempotencyPreserved
RequestRegistry-->>ERS: "Request Recorded Or Confirmed"
ERS->>CanonicalEntityRegistry: "Get Canonical URI By Mention URI"
alt CanonicalMappingExists
CanonicalEntityRegistry-->>ERS: "Canonical URI Found"
ERS->>EntityResolutionRESTAPI: "200 OK Canonical URI"
EntityResolutionRESTAPI->>Originator: "200 OK Canonical URI"
else CanonicalMappingMissing
CanonicalEntityRegistry-->>ERS: "No Canonical URI Found"
ERS->>ResolutionDecisionStore: "Create Preliminary Decision With New
Canonical URI"
ResolutionDecisionStore-->>ERS: "Preliminary Decision Recorded"
ERS->>CanonicalEntityRegistry: "Create Canonical Entity With Mention
And New Canonical URI"
```

```

ERS->>MessagingMiddleware: "Publish Resolution Request For ERE
Processing"

CanonicalEntityRegistry-->>ERS: "Canonical Entity Created And Mention
Linked"
alt NormalMinting
ERS->>EntityResolutionRESTAPI: "200 OK Newly Minted Canonical URI"
EntityResolutionRESTAPI->>Originator: "200 OK Newly Minted Canonical
URI"
else RepairMinting
ERS->>EntityResolutionRESTAPI: "202 Accepted Repaired New Canonical
URI"
EntityResolutionRESTAPI->>Originator: "202 Accepted Repaired New
Canonical URI"
end
end
end
end
alt StoreUnavailableOrInternalError
ERS->>EntityResolutionRESTAPI: "503 Service Unavailable"
EntityResolutionRESTAPI->>Originator: "503 Service Unavailable"
end

```

Originator asks for a lookup

[Diagram](#)

```

sequenceDiagram
    participant Originator
    participant EntityResolutionRESTAPI as "Entity Resolution REST API"
    participant ERS
    participant RequestRegistry as "Request Registry"
    participant CanonicalEntityRegistry as "Canonical Entity Registry"

    Originator->>EntityResolutionRESTAPI: "GET
/resolve/{originatorId}/{originatorRequestId}"
EntityResolutionRESTAPI->>ERS: "Validate Identifiers And Authorise"
ERS->>RequestRegistry: "Find Request Record(originatorId,
originatorRequestId)"

```

```
alt "Request not found"
RequestRegistry-->>ERS: "Not Found"
ERS->>EntityResolutionRESTAPI: "404 Not Found"
EntityResolutionRESTAPI->>Originator: "404 Not Found"
else "Request found"
RequestRegistry-->>ERS: "Request Found(canonicalId)"
ERS->>CanonicalEntityRegistry: "Get Canonical Entity By Canonical
Id(canonicalId)"

alt "Canonical entity missing (repair needed)"
CanonicalEntityRegistry-->>ERS: "Not Found"
ERS->>EntityResolutionRESTAPI: "409 Conflict(Registry Inconsistency)"
EntityResolutionRESTAPI->>Originator: "409 Conflict"
else "Canonical entity found"
CanonicalEntityRegistry-->>ERS: "Canonical Entity Found(canonicalUri)"
ERS->>EntityResolutionRESTAPI: "200 OK Canonical URI"
EntityResolutionRESTAPI->>Originator: "200 OK Canonical URI"
end
end

alt "Invalid identifiers or request"
ERS->>EntityResolutionRESTAPI: "400 Bad Request"
EntityResolutionRESTAPI->>Originator: "400 Bad Request"
end

alt "Store unavailable or internal error"
ERS->>EntityResolutionRESTAPI: "503 Service Unavailable"
EntityResolutionRESTAPI->>Originator: "503 Service Unavailable"
end
```

UC-B1.2

UC-B-1.2 — Integrate Resolution Results from ERE (Asynchronous)

Use Case Level

Blue (User-goal)

Primary Actor

Entity Resolution System (ERS)

Scope

ERS internal orchestration boundary towards Entity Resolution Engines (ERE)

Supporting Actors

- Entity Resolution Engine (ERE)
- Messaging Middleware (asynchronous pub/sub)
- Resolution Decision Store (authoritative governance store)
- Canonical Entity Registry (authoritative current-state projection)

Stakeholders and Interests

- **ERS**: requires a pluggable, engine-agnostic integration boundary with deterministic correlation, bounded decision context, and non-blocking failure handling.
- **ERE implementers**: require a stable and minimal contract: consume a resolution request and return candidate alignment links, without responsibility for ERS state or governance.
- **Operators**: require traceability of orchestration actions and outcomes through logs and telemetry, without coupling to specific engine implementations.

Brief Description

This use case describes the asynchronous integration of resolution results produced by an Entity Resolution Engine (ERE) into ERS governance state.

For an Entity Mention that has already been registered and assigned a deterministic canonical identifier (as defined in UC-B-1.1), ERS forwards a resolution request to one or more connected EREs using asynchronous messaging. The request contains the structured Entity Mention Representation and may include a list of previously rejected canonical identifiers that must not be proposed again for this mention.

The ERE processes the request independently and returns a set of candidate alignment links. ERS treats the ERE as a black box: ERE outputs are interpreted as proposals and never as authoritative decisions. Integration of these proposals may update the Resolution Decision and, if applicable, the current canonical assignment.

Trigger

ERS initiates asynchronous resolution when an Entity Mention requires engine-assisted evaluation, for example because:

- no prior resolution attempt exists for the mention, or
- a previous automated or manual decision explicitly requested re-resolution.

Preconditions

- The Entity Mention has been accepted and registered via the ERS public API.
- A default canonical identifier has been deterministically derived and recorded.
- A Resolution Decision exists for the mention with an initial governance state.
- Asynchronous messaging channels between ERS and ERE are available.

Success Guarantee (Postconditions)

When a resolution response is successfully received from an ERE, ERS:

- prunes the received candidate alignment links to the configured top N proposals;
- stores the pruned candidate set as decision context in the Resolution Decision Store;
- evaluates the top candidate against the configured auto-acceptance rules;
- updates the Resolution Decision status accordingly (e.g. automatically accepted or pending manual review);
- updates the Canonical Entity Registry if and only if the governance rules select a new canonical mapping.

The Canonical Entity Registry stores only the latest accepted canonical state. Historical context and candidate sets are retained exclusively in the Resolution Decision Store.

Minimal Guarantees (On Failure)

- If ERS cannot publish a request to the messaging middleware, the failure is logged and no decision state is changed.
- If an ERE returns an error response or no response is received, ERS retains the previously assigned canonical mapping.
- ERE failures never propagate to external API clients and do not invalidate existing canonical assignments.

Main Success Scenario

1. ERS prepares a resolution request for a single Entity Mention, including correlation identifiers and the structured Entity Mention Representation.
2. ERS publishes the request to the messaging middleware on the agreed request channel.
3. An ERE consumes the request and performs resolution using its internal logic.
4. The ERE publishes a resolution response containing a set of candidate alignment links.
5. ERS consumes the response and correlates it to the original mention.
6. ERS prunes the candidate set to the configured top N links.

7. ERS stores the pruned candidate set as decision context in the Resolution Decision Store.
8. ERS evaluates the top candidate against governance rules and updates the Resolution Decision status.
9. ERS updates the Canonical Entity Registry if the governance outcome selects a new canonical mapping.
10. ERS records essential orchestration actions and outcomes in the ERS action log.

Extensions

2a. Failure to publish resolution request

ERS logs the failure and performs no retry. The existing canonical assignment remains unchanged until a new resolution attempt is triggered externally.

4a. ERE returns an error response

ERS logs the error and does not update decision context or canonical state.

5a. Duplicate or late resolution response

ERS integrates the response if it is applicable; if multiple responses are correlated to the same mention, the latest applicable response (by agreed ordering rule) determines the decision context.

6a. Fewer than N candidate links returned

ERS stores all returned candidates.

6b. More than N candidate links returned

ERS stores only the top N candidates.

Special Requirements

- Asynchronous transport is mandatory; ERS–ERE interaction is non-blocking.
- Decision context is bounded to top N candidate links per mention.
- Correlation and deduplication are based on the same identifiers used for request registration.
- Observability of orchestration actions is mandatory; exact telemetry formats are out of scope.

Technology and Data Variations

- Messaging middleware may vary by deployment and is abstracted at the architectural level.
- Candidate confidence scores are treated as comparable numeric values by ERS; their semantics are engine-specific and not interpreted as probabilities.

Frequency of Occurrence

High; triggered for each mention that requires engine-assisted resolution.

Out of Scope

- Full rebuild protocols and historical reclustering.
- Manual curation actions.

- Retry strategies beyond explicit re-invocation.

Open Issues

- Precise request and response schemas for the ERS–ERE contract.
- Ordering rule for multiple responses (queue order versus explicit timestamps).
- Lifecycle and revocation semantics for rejected canonical identifiers.

UC-B1.2 diagram

ERE resolves a request

[Diagram](#)

sequenceDiagram

```
participant ERS as "Entity Resolution Service (ERS)"
participant MessagingMiddleware as "Messaging Middleware"
participant ERE as "Entity Resolution Engine (ERE)"
participant ResolutionDecisionStore as "Resolution Decision Store"
participant CanonicalEntityRegistry as "Canonical Entity Registry"

ERS->>MessagingMiddleware: "Publish resolution request(originatorId,
originatorRequestId, entityMention, draftCanonicalId,
rejectedCanonicalIds)"
MessagingMiddleware->>ERE: "Deliver resolution request"

ERE->>ERE: "Execute entity resolution"
ERE->>MessagingMiddleware: "Publish resolution
result(alignmentLinkSet)"

MessagingMiddleware->>ERS: "Deliver resolution
result(alignmentLinkSet)"

ERS->>ERS: "Prune alignment link set (top N)"
ERS->>ERS: "Select automatic alignment link"

ERS->>ResolutionDecisionStore: "Upsert resolution
decision(originatorId, originatorRequestId, entityMention,
alignmentLinkSet)"
ResolutionDecisionStore-->>ERS: "Decision recorded"

ERS->>CanonicalEntityRegistry: "Apply decision to canonical
registry(entityMention, canonicalEntityId)"
CanonicalEntityRegistry-->>ERS: "Canonical registry updated"

alt "Resolution engine error"
MessagingMiddleware->>ERS: "Deliver resolution error(originatorId,
originatorRequestId)"
ERS->>ResolutionDecisionStore: "Record failed decision"
end

alt "Duplicate resolution result"
```

```
ERS->>ResolutionDecisionStore: "Detect existing decision(originatorId,
originatorRequestId)"
ResolutionDecisionStore-->>ERS: "Decision already applied"
end

alt "Store unavailable or internal error"
ERS->>ResolutionDecisionStore: "Record deferred decision"
end
```

UC-B1.3

UC-B-1.3 — Lookup Canonical Identifier via ERS API (Read-only)

Use Case Level

Blue (User-goal)

Primary Actor

API Client (e.g. TED-SWS, Link Curation Backend, or any authorised originator)

Scope

Entity Resolution System (ERSys) — Public API (Lookup)

Supporting Actors

- Request Registry (authoritative request record store)
- Canonical Entity Registry (authoritative current-state projection)

Stakeholders and Interests

- **API Client:** wants a read-only, deterministic way to retrieve the current canonical identifier for a previously submitted request, without triggering re-resolution or other side effects.
- **ERS operators:** require bounded execution, stable idempotent semantics, and clear error signalling.
- **Downstream consumers:** depend on canonical identifiers that may be refined over time (via curation or rebuild), and need a safe mechanism to observe the latest accepted state.

Brief Description

The API Client queries ERS for the current canonical identifier associated with a previously submitted (`originatorId`, `originatorRequestId`[, `entityType`]) request.

This operation is strictly read-only. It does not trigger resolution, re-resolution, curation, or rebuild. It returns the latest accepted canonical identifier as recorded in the Canonical Entity Registry, correlated to an existing Request Registry record.

This use case supports client-side reconciliation: clients may submit mentions via UC-B-1.1 and later perform lookups to observe whether the current canonical assignment has been refined by asynchronous resolution, manual curation, or rebuild operations.

Trigger

The API Client submits a lookup request for a previously submitted correlation key.

Preconditions

- The API Client is authorised to call the ERS lookup API.

- The lookup request provides the required correlation identifiers: `originatorId` and `originatorRequestId`. If `entityType` is part of the correlation policy, it is also provided.

Success Guarantee (Postconditions)

For a successful lookup:

- ERS returns the current canonical identifier associated with the correlation key.
- ERS does not modify any state.
- ERS responds within bounded time.

Minimal Guarantees (On Failure)

- If the correlation key has no matching request record, ERS returns a “not found” outcome.
- If the request record exists but no canonical mapping is available due to an internal inconsistency, ERS returns an explicit service error.
- If multiple conflicting request records exist for the same correlation key (should not happen if idempotency rules were enforced), ERS returns an explicit conflict error.

Main Success Scenario

1. The API Client submits a lookup request providing (`originatorId`, `originatorRequestId`[, `entityType`]).
2. ERS validates the request format and authorisation.
3. ERS checks the Request Registry for an existing request record matching the correlation key.
4. ERS retrieves the current canonical identifier from the Canonical Entity Registry.
5. ERS returns the canonical identifier to the API Client.

Extensions

3a. Request record not found

If no request record exists for the correlation key, ERS returns a `NOT_FOUND` outcome.

3b. Idempotency conflict detected

If the Request Registry indicates conflicting records for the same correlation key, ERS returns a `CONFLICT` outcome.

4a. Canonical mapping missing despite existing request record

ERS returns an explicit service error and records the inconsistency for operational investigation.

Special Requirements

- Read-only semantics: the lookup operation does not create, update, or trigger any processing.
- Correlation: the lookup key is the same as used for request registration (UC-B-1.1).
- Bounded response time: no dependency on ERE processing.

Technology and Data Variations

- The lookup API may be expressed as a REST GET operation with correlation identifiers in the path or query parameters.

Frequency of Occurrence

Medium to high; expected to be used by clients performing delayed reconciliation.

Notes / Business Rules

- The returned canonical identifier represents the latest accepted governance state and may differ from the identifier observed immediately after submission if curation or rebuild operations have changed the accepted alignment.
- The lookup operation reflects the outcome of the currently accepted Resolution Decision, without exposing decision details.

Open Issues

- Whether entityType is mandatory in the lookup key or implicit in the request record.
- Exact external error code mapping (HTTP status codes versus domain categories).

UC-W2

UC-W-02 — Curate Resolution Decisions

Use Case Level

White (Summary)

Primary Actor

Curator (human user)

Scope

Entity Resolution System (ERSys) and Link Curation Application

Stakeholders and Interests

- **Curators:** need a clear, authoritative way to review and override automated resolution outcomes in order to ensure correctness and governance compliance.
- **Data governance owners:** require that manual decisions are traceable, authoritative, and consistently applied.
- **Downstream consumers:** depend on stable canonical identifiers that reflect approved governance decisions.
- **ERS operators:** require that manual curation integrates cleanly with automated resolution without breaking system invariants.

Brief Description

The Curator reviews existing Resolution Decisions produced by ERS and performs manual curation actions to govern how Entity Mentions are associated with canonical identifiers.

Manual curation operates exclusively at the level of Resolution Decisions. Curator actions update the governance state of a decision and take precedence over any automated proposals produced by Entity Resolution Engines. ERS projects the outcome of the current Resolution Decision into the Canonical Entity Registry.

Curation does not create new canonical identifiers. It selects among existing canonical identifiers or confirms the default assignment previously derived by ERS.

Trigger

A Curator accesses the Link Curation application to review and act upon one or more Resolution Decisions.

Preconditions

- The Curator is authenticated and authorised to perform curation actions.
- One or more Resolution Decisions exist for review, created via automated resolution or default assignment.

Success Guarantee (Postconditions)

For each curated Resolution Decision:

- The governance state of the Resolution Decision is updated according to the Curator's action.
- Manual curation overrides any automated assessment for that decision.
- The Canonical Entity Registry is updated to reflect the canonical mapping governed by the updated Resolution Decision.
- The updated state is immediately visible to lookup operations and downstream consumers.

Minimal Guarantees (On Failure)

- If a curation action cannot be persisted, the existing Resolution Decision and canonical mapping remain unchanged.
- Partial failures do not leave Resolution Decisions in an inconsistent state.

Main Success Scenario

1. The Curator selects a Resolution Decision for review in the Link Curation application.
2. ERS presents the current decision state and relevant context for that decision.
3. The Curator performs a manual curation action on the Resolution Decision.
4. ERS validates and records the updated governance state of the Resolution Decision.
5. ERS updates the Canonical Entity Registry to reflect the curated canonical mapping.
6. The updated canonical assignment becomes effective immediately for subsequent lookups and downstream processing.

Extensions

2a. Resolution Decision not found or no longer available

ERS reports that the selected decision cannot be curated.

3a. Concurrent modification detected

ERS rejects the curation action and reports a conflict.

Special Requirements

- Manual curation actions are authoritative and dominate automated resolution outcomes.
- Curation affects governance state only and does not modify deterministic identifier derivation rules.
- Curation actions are traceable.

Frequency of Occurrence

Medium; driven by data quality workflows and governance needs.

Notes / Business Rules

- Manual curation establishes the authoritative canonical mapping until explicitly changed by a subsequent curation or rebuild operation.
-

UC-B2.1

UC-B-2.1 — Review and Decide on a Resolution Decision

Use Case Level

Blue (User-goal)

Primary Actor

Curator (human user)

Scope

Link Curation application and ERS curation API, including updates to the Resolution Decision Store and the Canonical Entity Registry for a single Entity Mention.

Supporting Actors

- Entity Resolution System (ERS)

Goal

Allow a Curator to review a Resolution Decision that requires manual oversight and to authoritatively determine how the corresponding Entity Mention is governed and projected into the Canonical Entity Registry.

Trigger

The Curator selects a Resolution Decision with status `PendingManualReview` from the decision list in the Link Curation application.

Preconditions

- A Resolution Decision exists with status `PendingManualReview`.
- The Resolution Decision is visible in the Link Curation application.
- The Resolution Decision contains a complete decision context, including:
 - the subject Entity Mention,
 - one proposed canonical mapping (top-ranked automated proposal),
 - zero or more alternative candidate mappings.
- The Curator is authenticated and authorised to perform curation actions.

Success Guarantees (Postconditions)

For a successful curation action:

- The Curator's selected outcome is recorded as the authoritative governance state of the Resolution Decision.
- Manual curation overrides any automated proposal for that decision.
- The Canonical Entity Registry is updated to reflect the canonical mapping governed by the updated Resolution Decision.

- The Resolution Decision is marked as reviewed and removed from the curation queue.
- The curation action is recorded via standard system logging.

Minimal Guarantees (On Failure)

- If the system cannot persist the Curator's decision or update canonical state, no partial changes are applied.
- The Resolution Decision remains in status **PendingManualReview** and visible in the curation queue.

Main Success Scenario

1. The Curator opens the list of Resolution Decisions pending manual review.
2. The Curator selects a single Resolution Decision.
3. The system presents the decision context, including:
 - the Entity Mention representation,
 - the currently proposed canonical mapping,
 - alternative candidate mappings.
4. The Curator selects one of the supported curation actions.
5. The Curator submits the curation decision.
6. The system validates that the Resolution Decision is still eligible for curation.
7. The system records the updated governance state of the Resolution Decision.
8. The system updates the Canonical Entity Registry to reflect the curated canonical mapping.
9. The Resolution Decision is removed from the curation queue.

Extensions

E1. Resolution Decision no longer eligible

If the Resolution Decision has already been reviewed or superseded, the system rejects the submission and informs the Curator.

E2. Assign alternative canonical mapping

The Curator selects one of the alternative candidate mappings. The selected mapping becomes authoritative; other candidates are marked as rejected in the decision context.

E3. Reject all candidate mappings

All candidate mappings in the decision context are marked as rejected. The default canonical mapping derived by ERS remains authoritative. A new asynchronous resolution attempt may be triggered with rejected mappings excluded.

E4. Concurrent curation submission

If another Curator submits a decision for the same Entity Mention concurrently, the system detects the conflict and rejects the later submission.

E5. Canonical registry update failure

If the system fails to update the Canonical Entity Registry, the entire curation operation is rolled back and the Resolution Decision remains pending.

Special Requirements

- Only Resolution Decisions with status **PendingManualReview** are eligible for this use case.
- Exactly one authoritative governance outcome exists per Entity Mention at any time.
- Manual curation actions do not create new canonical identifiers and do not alter deterministic identifier derivation rules.
- Curation actions are auditable and traceable.

Notes / Business Rules

- Canonical Entity Registry stores only the latest accepted canonical mapping; rejected alternatives and decision context are retained exclusively in the Resolution Decision Store.
- During rebuild operations, curated decisions may be superseded, but curator-confirmed constraints remain authoritative inputs.

Open Issues

- None at this stage.

UC-B2.2

UC-B-2.2 — Perform Bulk Curation Actions

Use Case Level

Blue (User-goal)

Primary Actor

Curator (human user)

Scope

Link Curation application and ERS curation API, limited to bulk operations initiated from the decision list view, including updates to the Resolution Decision Store and the Canonical Entity Registry.

Supporting Actors

- Entity Resolution System (ERS)

Goal

Allow a Curator to efficiently resolve multiple Resolution Decisions requiring manual oversight by applying a uniform governance action, without inspecting each decision in full detail.

Trigger

The Curator selects multiple Resolution Decisions with status **PendingManualReview** from the decision list and initiates a bulk curation action.

Preconditions

- One or more Resolution Decisions exist with status **PendingManualReview**.
- Each selected Resolution Decision contains a decision context with a proposed canonical mapping.
- The Curator is authenticated and authorised to perform bulk curation actions.

Success Guarantees (Postconditions)

For each successfully processed Resolution Decision:

- The Curator's action is recorded as the authoritative governance outcome of the Resolution Decision, together with curator identity and timestamp.
- The Canonical Entity Registry is updated to reflect the canonical mapping governed by the updated Resolution Decision, where applicable.
- The Resolution Decision is marked as reviewed and removed from the curation queue.
- A lightweight audit log entry describing the action is recorded.

Minimal Guarantees (On Failure)

- If one or more selected Resolution Decisions cannot be processed due to validation, persistence, or consistency errors, those decisions remain in status **PendingManualReview**.
- Other selected Resolution Decisions may still be processed successfully.
- No partial or inconsistent canonical state is made visible for any individual decision.

Main Success Scenario

1. The Curator opens the list of Resolution Decisions pending manual review.
2. The Curator selects multiple Resolution Decisions from the list.
3. The system presents the supported bulk curation actions.
4. The Curator selects a bulk action and submits it.
5. For each selected Resolution Decision, the system:
 - validates that the decision is still eligible for curation,
 - records the Curator's governance outcome,
 - updates the Canonical Entity Registry as governed by the updated Resolution Decision,
 - marks the Resolution Decision as reviewed.
6. Successfully processed Resolution Decisions are removed from the curation queue.

Extensions

E1. Partial failure during bulk processing

If an internal error occurs while processing one or more Resolution Decisions, those decisions remain pending, while others may complete successfully.

E2. Concurrent curation detected

If another Curator has already modified a Resolution Decision, the system rejects the bulk operation for that decision and reports the conflict. No last-write-wins policy is applied.

E3. Reject all candidates (bulk)

For each affected Resolution Decision, all proposed candidate mappings are marked as rejected. The default canonical mapping derived by ERS remains authoritative. A new asynchronous resolution attempt may be triggered with rejected mappings excluded.

Special Requirements

- Only Resolution Decisions with status **PendingManualReview** are eligible for bulk curation.
- Bulk curation supports a restricted set of actions (e.g. accept proposed mapping, reject all candidates).
- Assigning alternative canonical mappings is not permitted in bulk mode.
- Each Resolution Decision is processed atomically: updates to the Resolution Decision Store and the Canonical Entity Registry are committed together or not at all.
- Bulk curation supports partial success; failure or conflict affecting one decision does not prevent processing of others.

Notes / Business Rules

- Manual bulk curation actions are authoritative and override automated proposals for the affected Resolution Decisions.
- Any subsequent re-resolution triggered by bulk actions is asynchronous and not visible in the curation UI.
- UI previews shown in bulk selection mode are indicative only and do not expose full decision context or internal scoring semantics.

Frequency of Occurrence

Medium; driven by backlog management and operational efficiency needs.

Open Issues

- Exact set of supported bulk actions and corresponding UI affordances.

UC-W3

UC-W-03 — Rebuild Resolution State

Use Case Level

White (Summary)

Primary Actor

System Administrator

Stakeholders and Interests

- **System Administrators / ERS Operators:** require a controlled, explicit mechanism to recover from systemic resolution errors, degraded clustering quality, or major configuration changes, while limiting operational risk and ensuring predictable behaviour.
- **Curators:** expect that manually curated decisions remain authoritative and are not silently overridden or lost during corrective operations.
- **Data Consumers:** depend on continuity and long-term stability of canonical identifiers, even when large-scale corrective actions are applied.
- **Data Governance Owners:** require transparency, traceability, and clear responsibility boundaries for disruptive and potentially irreversible corrective actions.
- **ERS Maintainers:** need rebuild semantics that are isolated from normal incremental resolution flows and do not undermine architectural invariants.

Brief Description

This use case describes a system-level corrective capability that allows an authorised administrator to trigger a controlled rebuild of the resolution state for a single selected entity type.

A rebuild is an explicit, exceptional operation that reprocesses all known Entity Mentions of the selected entity type by generating new automated resolution proposals and integrating them through the normal decision-centric governance model. Rebuild operations do not change canonical identifier derivation rules and do not bypass Resolution Decisions. Instead, they regenerate decision context at scale, while preserving authoritative manual curation constraints.

Rebuild operations are intentionally scoped to one entity type at a time to limit operational blast radius, preserve the stability of unaffected canonical domains, and align corrective action with the specific quality, configuration, or governance concerns that motivated the rebuild. Rebuilds are disruptive and long-running, but are designed to be transparent, traceable, and operationally isolated from normal incremental resolution flows.

Goal / Value

Restore or improve the global coherence and internal consistency of resolution outcomes for a selected entity type by regenerating and re-integrating automated proposals under a controlled rebuild operation, while preserving governed knowledge, deterministic identity rules, and downstream system stability.

Scope (System-of-Interest)

The Entity Resolution System (ERS), including:

- regeneration of automated resolution proposals for the selected entity type,
- integration of rebuild outcomes via Resolution Decisions,
- maintenance of the Canonical Entity Registry as a projection of the currently governed state.

Out of Scope

- Re-resolution of a single Entity Mention via normal APIs.
- Localised corrective actions limited to a single canonical cluster.
- External, client-triggered rebuild requests.
- Administrative tooling outside the Link Curation application (e.g. ad-hoc scripts or CLI-based recovery).

Trigger

An authorised System Administrator explicitly initiates a rebuild operation for a specific entity type, typically in response to detected systemic resolution issues, degraded clustering quality, or significant configuration changes.

Preconditions

- The System Administrator is authenticated and authorised to initiate rebuild operations.
- No other rebuild operation is currently active.
- The selected entity type is in a stable operational state suitable for rebuild initiation.

Success Guarantees (Postconditions)

For the selected entity type:

- All known Entity Mentions remain associated with exactly one canonical identifier, in accordance with deterministic derivation rules.
- Existing manual curation constraints remain authoritative and are preserved throughout the rebuild.
- Resolution Decisions governing mention-to-canonical mappings may be superseded by rebuild outcomes, except where constrained by manual curation.
- The Canonical Entity Registry reflects the currently governed state resulting from rebuild integration.
- The entity type returns to normal incremental resolution mode after rebuild completion or accepted partial completion.

Minimal Guarantees (On Failure)

- All successfully integrated Resolution Decisions applied prior to failure remain effective.
- No partial or inconsistent canonical state is exposed.
- Canonical identifier derivation rules remain unchanged.
- The entity type is returned to normal incremental resolution mode.

- The administrator may re-initiate the rebuild after corrective action.

Notes / Business Rules

- At most one rebuild operation may be active at any time, and it applies to exactly one entity type.
- During a rebuild, normal resolution and curation operations remain available; their outcomes may be superseded by rebuild-generated decisions for the affected entity type.
- Rebuild operations operate exclusively through the Resolution Decision model and do not directly mutate canonical state outside of governed projections.
- Rebuild behaviour (e.g. state-aware versus destructive regeneration of automated proposals) is a system configuration decision; state-aware rebuild is recommended for production environments.

Frequency of Occurrence

Low; rebuild operations are exceptional and expected to be invoked rarely.

Open Issues

- Criteria for determining rebuild completion versus accepted partial completion.
- Operator guidance for coordinating rebuilds with ongoing curation activity.

UC-W4 (moved to Sec7)

UC-W4: Consult Resolution Statistics

Level: White (Summary)

Primary Actor: Curator (human user)

Stakeholders and Interests

- **Curators:** Want a quick, reliable overview of the current resolution and curation workload in order to prioritise review efforts and understand progress.
- **System Administrators / ERS Operators:** Need lightweight observability to assess system behaviour, backlog evolution, and the balance between automated and manual decisions.
- **Data Governance Owners:** Require high-level indicators demonstrating that governance processes are active, proportionate, and effective.
- **Data Consumers:** Indirectly depend on stable resolution quality, which is reflected in aggregate statistics rather than individual decisions.

Brief Description

This use case describes how a human curator consults aggregated, read-only statistics about the current state of entity resolution and curation through the Link Curation application. The system provides high-level indicators, scoped by entity type and optionally by time window, such as the volume of pending and resolved decisions, the proportion of automated versus manually curated outcomes, and the overall size of the canonical entity set.

The use case is intentionally informational and non-intrusive. It supports situational awareness, prioritisation, and governance oversight without exposing individual entities or decisions, and without allowing any modification of resolution state.

Goal / Value

Provide a simple, aggregated view of resolution and curation statistics so that human operators can understand the current state of the Entity Resolution System without inspecting individual decisions or entities.

Scope (System-of-Interest)

The Link Curation application and the Entity Resolution Service (ERS), limited to the retrieval and presentation of aggregated, read-only statistics derived from existing operational stores.

Implementation note (non-normative): statistics are derived from the Decision Store and the Canonical Entity Registry.

Out of Scope

- Inspection or modification of individual decisions or entities.
- Operational monitoring, alerting, or performance metrics.

- Historical trend analysis or time-series dashboards.
- Administrative or corrective actions.

Trigger

A curator explicitly requests to view resolution statistics, typically to assess current workload, progress, or overall resolution health.

Preconditions

- The curator is authenticated and authorised to access statistical views.
- The ERS statistics endpoint is available.

Success Guarantees (Postconditions)

- The user can retrieve aggregated statistics scoped by entity type and, where applicable, by time frame.
- Statistics reflect the current observable state of the system at the time of the request.
- The operation is strictly read-only and does not alter any system state.

Minimal Guarantees (On Failure)

- No system state is modified.
- The user is informed that statistics are temporarily unavailable.

Notes / Business Rules

- Statistics are derived from existing operational stores and are not independently curated or corrected.
- Counts are indicative and may change as resolution and curation activities continue.
- Statistics may be temporarily inconsistent during long-running rebuild operations.
- Statistics are indicative operational signals and must not be interpreted as audit evidence or compliance artefacts.

UC-B4.1

UC-B-4.1: View Resolution and Curation Statistics

- **Level:** Blue (User-goal)
- **Primary Actor:** Curator (human user)
- **Supporting Actors:**
 - Entity Resolution Service (ERS)

Scope (System-of-Interest)

Link Curation application and ERS statistics API, limited to read-only retrieval of aggregated statistics derived from the Decision Store and the Canonical Entity Registry.

Goal

Allow a curator to obtain a concise, aggregated overview of resolution and curation activity in order to understand backlog, progress, and system state without inspecting individual decisions.

Trigger

The curator opens the statistics view in the Link Curation application.

Preconditions

- The curator is authenticated and authorised to access the Link Curation application.

Success Guarantees (Postconditions)

- Aggregated statistics are displayed to the curator.
- Statistics are scoped by entity type and an optional timeframe.
- If no timeframe is specified, statistics reflect **all available time**.
- No system state is modified.

Minimal Guarantees (On Failure)

- If statistics cannot be retrieved, no system state is modified.
- The curator is informed that statistics are temporarily unavailable.

Main Success Scenario (Happy Path)

1. The curator navigates to the statistics section of the Link Curation application.
2. The curator optionally selects an entity type.
3. The curator optionally specifies a timeframe.
4. The system retrieves aggregated statistics from the ERS.
5. The system presents the statistics, including:

- registry-level statistics (e.g. number of canonical entities, entity mentions, resolution requests),
- curation-level statistics (e.g. pending, reviewed, and automatic decisions).

Extensions (Alternative / Error Paths)

- **E1 (from step 4): Statistics service unavailable**
 - The system fails to retrieve statistics from the ERS.
 - An error message is displayed to the curator.
- **E2 (from step 3): Invalid timeframe specification**
 - The system rejects the timeframe input.
 - The curator is prompted to correct or clear the timeframe.

Minimal Internal State Notes

- Statistics are computed on demand from operational stores.
- Counts may change between successive requests due to ongoing resolution and curation activity.

Notes / Business Rules

- Statistics are indicative and intended for situational awareness, not auditing.
- Statistics may temporarily reflect mixed states during rebuild operations.
- No guarantees are made about historical completeness or transactional consistency.

Open Questions

- None at this stage.

UC-W5

UC-W-5: Manage Curation Users

- **Level:** White (Summary)
- **Primary Actor:** System Administrator

Stakeholders and Interests

- **System Administrator:** Needs a simple, reliable way to provision and control curator access, ensuring only authorised users can curate decisions.
- **Curators:** Need timely account provisioning, predictable access, and clear suspension/reactivation handling.
- **ERS Operators:** Need operational control that does not require external identity tooling, while remaining compatible with optional SSO integration.
- **Data Governance Owners:** Need assurance that only authorised curators can apply governance actions affecting canonical mappings.

Goal / Value

Provide minimal but sufficient user administration for the Link Curation application so that curator access can be created, updated, suspended/reactivated, and retired in a controlled and reversible way.

Scope (System-of-Interest)

Link Curation application and ERS User Store / User Management API, including:

- creation and maintenance of curator accounts,
- account lifecycle state management (active/suspended/deactivated),
- password reset with forced change on next login,
- immediate revocation of active sessions/tokens upon suspension or deactivation.

Out of Scope

- Identity governance processes (approvals, role mining, recertification).
- Advanced audit trails beyond standard system logging.
- External Identity Provider (IdP) / SSO integration specifics (treated as optional and outside the boundary).
- Fine-grained authorisation beyond the two roles: **Admin** and **Curator**.

Success Guarantees

- The administrator can create, update, suspend/activate, soft-delete, and reset passwords for curator accounts.
- Curator identity is uniquely keyed by **email**.
- Suspended/deactivated users have their access revoked immediately (active sessions/tokens invalidated).

- Soft-deleted users remain retained for reference (e.g. past decisions), but cannot authenticate.

Minimal Guarantees (On Failure)

- If a user management operation fails, no partial or ambiguous user state is exposed.
- The system reports the failure to the administrator and does not grant unintended access.

Brief Description

This use case describes the minimal user management capability for the Link Curation application. A system administrator manages curator accounts using an internal user store and API, independent of any external identity provider. The administrator can provision and maintain users identified by email, suspend or reactivate accounts, soft-delete accounts while retaining historical references, and reset passwords by issuing a temporary password that must be changed at next login. For safety, any suspension or deactivation immediately revokes active sessions and tokens.

The concrete administrative procedure and tooling for managing access may be realised via SSO configuration and operational processes and is not specified as a Blue use case.

UC-W6

UC-W-06 — Trace ERS Operations End-to-End

Use Case Level

White (Summary)

Primary Actor

Operator (DevOps, SRE, or Developer)

Stakeholders and Interests

- **Operators:** need rapid diagnosis of failures, latency hotspots, and cross-component issues in a distributed, asynchronous system.
- **Developers:** need end-to-end correlation of ERS activity across APIs, orchestration logic, persistence layers, and message-driven interactions to support debugging and maintenance.
- **Client Systems:** benefit indirectly from reduced incident resolution time and clearer fault localisation.
- **Security and Governance Owners:** require traceability signals that support operational oversight without exposing sensitive entity content.

Goal / Value

Provide best-effort, metadata-only, end-to-end traceability of key ERS operations so that execution paths can be correlated across synchronous and asynchronous boundaries without affecting business behaviour or governance guarantees.

Scope (System-of-Interest)

The Entity Resolution Service (ERS), including:

- public ERS API operations (e.g. resolve, lookup, bulk variants),
- Link Curation API operations affecting Resolution Decisions and canonical state,
- internal ERS business operations interacting with the Request Registry, Decision Store, and Canonical Entity Registry,
- propagation of trace context across the ERS↔ERE asynchronous messaging boundary.

Trace emission by the Entity Resolution Engine (ERE) beyond context propagation is optional and treated as best-effort.

Out of Scope

- Logging or tracing of entity payload content or large free-text fields.
- Prescribing a specific tracing backend, storage system, or APM vendor.
- Guaranteeing availability or durability of external telemetry infrastructure.
- Using trace data to drive or alter business decisions.

Success Guarantees

- For each traced ERS operation, the system emits OpenTelemetry-compatible trace signals containing correlation metadata and error indicators where applicable.
- Trace context is propagated across asynchronous ERS↔ERE message exchanges so that related operations can be correlated when downstream components participate.
- Absence, delay, or loss of telemetry signals does not block or alter ERS business operations.

Minimal Guarantees (On Failure)

- If telemetry infrastructure is unavailable or overloaded, business operations continue unaffected.
- Partial traces or missing spans may occur and are acceptable.

Brief Description

This use case describes the observability capability of ERS that allows operators and developers to follow the execution of resolution, curation, and rebuild-related operations across service boundaries.

ERS emits trace metadata for key operations and propagates trace context across asynchronous message queues to support correlation across distributed components. Trace data is intentionally limited to identifiers and categorical metadata and is not used as an input to governance, resolution decisions, or client-visible behaviour.

Notes / Business Rules

- Telemetry follows best-effort semantics and must never become a prerequisite for correct system operation.
- Trace data is minimised to avoid leakage of sensitive or high-volume information.
- Sampling policies are configuration-driven and may vary by environment; error and failure traces are expected to be prioritised.
- ERS traces remain valid even if downstream components, including ERE, do not emit spans.

Open Questions

- None at this level; transport-specific propagation details are addressed in contracts and implementation guidance.

UC-W7 — Validate and Activate ERS Configuration

UC-W-07 — Validate and Activate ERS Configuration

Use Case Level

White (Summary)

Primary Actor

System Administrator (Operator)

Stakeholders and Interests

- **System Administrator / Operator:** needs predictable startup behaviour, clear validation failures, and safe deployment through static configuration.
- **API Clients:** rely on stable, well-defined behaviour for supported entity types, timeouts, and request validation.
- **Curators:** depend on consistent behaviour of configured entity types and curation workflows.
- **Developers / Maintainers:** require configuration-driven variability without code changes, while avoiding hidden coupling or configuration drift between ERS and ERE.

Goal / Value

Enable deployment of the Entity Resolution System in multiple operational contexts by externalising supported variability into validated, static configuration, while ensuring that each configured variant is coherent, explicit, and safe to run.

Scope (System-of-Interest)

ERS configuration loading, validation, and activation at startup, including:

- declaration of supported entity types,
- routing of entity types to resolution engines (ERE entrypoints or channels),
- configuration of ERS↔ERE messaging endpoints,
- client-facing timeout budgets and operational limits relevant to request handling.

Configuration is treated as static-at-startup; applying changes requires a controlled restart.

Out of Scope

- Runtime or hot reloading of configuration.
- UI-based configuration editing.
- Configuration governance workflows (approvals, versioning, promotion between environments).
- Engine-internal behaviour, algorithms, or scoring logic.

Success Guarantees

- ERS supports multiple deployment variants by allowing supported entity types, engine routing, messaging endpoints, and operational limits to be configured without code changes.
- On startup, ERS validates the complete configuration and either:
 - starts successfully with the configured variant activated, or
 - fails fast with explicit, actionable validation errors.
- At runtime, requests referencing unsupported or inactive entity types are rejected as invalid requests.

Minimal Guarantees (On Failure)

- If configuration validation fails, ERS does not start and does not expose partial or ambiguous behaviour.
- No runtime operation proceeds under an incomplete or inconsistent configuration.

Brief Description

This use case describes how ERS externalises permitted variability into configuration that is loaded and validated at startup. Configuration defines which entity types are supported, how they are routed to resolution engines, and which operational limits apply to client-facing interactions.

ERS applies a fail-fast principle: configuration errors are detected early and prevent activation. Once started, ERS operates strictly according to the activated configuration until the next controlled restart.

Configuration values that may be reused across ERS and ERE deployments are treated as compatibility hints only and must not introduce runtime coupling or shared state between systems.

Notes / Business Rules

- Configuration is authoritative only for structural and operational parameters, not for governance semantics defined by ADRs.
- Validation covers syntactic and semantic correctness; external connectivity checks are implementation-dependent.
- Optional environment-based overrides, if supported, must be validated under the same rules and must not allow partial activation.

Open Questions

- Canonical configuration format and packaging strategy (e.g. single bundle vs per-entity-type files).
- Whether configuration validation includes optional connectivity checks to messaging infrastructure at startup.